

I'll implement a comprehensive A2A agent card system with unique skill IDs, task management, and hierarchical agent IDs. Here's the complete implementation:

1. Agent Card System Implementation

1.1 Agent Card Schema & Management

```
```python
a2a/agent_cards.py
import uuid
from datetime import datetime
from typing import Dict, List, Optional, Set
from enum import Enum
from dataclasses import dataclass, field
import json

class MediaType(Enum):
 TEXT_PLAIN = "text/plain"
 APPLICATION_JSON = "application/json"
 APPLICATION_XML = "application/xml"
 TEXT_HTML = "text/html"
 APPLICATION_YAML = "application/yaml"
 APPLICATION_OCTET_STREAM = "application/octet-stream"
 IMAGE_PNG = "image/png"
 IMAGE_JPEG = "image/jpeg"
 APPLICATION_PDF = "application/pdf"

class TaskStatus(Enum):
 CREATED = "created"
 IN_PROGRESS = "in_progress"
 SUCCESS = "success"
 FAILURE = "failure"
 COMPLETE = "complete"
 CANCELLED = "cancelled"

class FlowDirection(Enum):
 FORWARD = "forward" # Normal task flow
 BACKWARD = "backward" # Rollback/compensation flow
 LATERAL = "lateral" # Peer-to-peer communication
 UPSTREAM = "upstream" # To higher-level agents
 DOWNSTREAM = "downstream" # To lower-level agents

@dataclass
class Skill:
```

```

"""Unique skill definition for agent capabilities"""
skill_id: str # Format: SKL-{type}-{uuid}
name: str # Task name
description: str # Detailed description
keywords: List[str] # Categorization keywords
sample_prompt: str # Example usage
supported_media: List[MediaType] # Supported media types
input_schema: Optional[Dict] = None # Expected input structure
output_schema: Optional[Dict] = None # Expected output structure
version: str = "1.0.0"
created: datetime = field(default_factory=datetime.utcnow)
last_updated: datetime = field(default_factory=datetime.utcnow)

```

@dataclass

class AgentCard:

```

"""A2A Agent Card defining capabilities and identity"""
agent_id: str # Hierarchical agent ID
parent_agent_id: Optional[str] = None # Parent agent for sub-agents
agent_type: str = "primary" # primary, sub_agent (sba), supervisor
skills: Dict[str, Skill] = field(default_factory=dict) # skill_id -> Skill
supported_protocols: List[str] = field(default_factory=lambda: ["A2A", "ACP"])
capabilities: List[str] = field(default_factory=list)
limitations: List[str] = field(default_factory=list)
security_clearance: str = "LEVEL_1"
created: datetime = field(default_factory=datetime.utcnow)
last_heartbeat: datetime = field(default_factory=datetime.utcnow)

```

def to\_json(self) -> Dict:

```

"""Convert agent card to JSON format for A2A communication"""
return {
 "agent_id": self.agent_id,
 "parent_agent_id": self.parent_agent_id,
 "agent_type": self.agent_type,
 "skills": {
 skill_id: {
 "skill_id": skill.skill_id,
 "name": skill.name,
 "description": skill.description,
 "keywords": skill.keywords,
 "sample_prompt": skill.sample_prompt,
 "supported_media": [media.value for media in skill.supported_media],
 "input_schema": skill.input_schema,
 "output_schema": skill.output_schema,
 "version": skill.version

```

```

 } for skill_id, skill in self.skills.items()
 },
 "supported_protocols": self.supported_protocols,
 "capabilities": self.capabilities,
 "limitations": self.limitations,
 "security_clearance": self.security_clearance,
 "created": self.created.isoformat(),
 "last_heartbeat": self.last_heartbeat.isoformat()
}

```

class SkillRegistry:

"""Manages skill creation and registration"""

def \_\_init\_\_(self):

self.skills: Dict[str, Skill] = {}

self.skill\_categories: Dict[str, Set[str]] = {} # category -> set of skill\_ids

def generate\_skill\_id(self, skill\_type: str) -> str:

"""Generate unique skill ID"""

unique\_id = str(uuid.uuid4())[:8]

return f"SKL-{{skill\_type.upper()}}-{{unique\_id}}"

def register\_skill(self, name: str, description: str, keywords: List[str],

sample\_prompt: str, supported\_media: List[MediaType],

skill\_type: str = "generic",

input\_schema: Optional[Dict] = None,

output\_schema: Optional[Dict] = None) -> Skill:

"""Register a new skill"""

skill\_id = self.generate\_skill\_id(skill\_type)

skill = Skill(

skill\_id=skill\_id,

name=name,

description=description,

keywords=keywords,

sample\_prompt=sample\_prompt,

supported\_media=supported\_media,

input\_schema=input\_schema,

output\_schema=output\_schema

)

self.skills[skill\_id] = skill

```

 # Categorize by keywords
 for keyword in keywords:
 if keyword not in self.skill_categories:
 self.skill_categories[keyword] = set()
 self.skill_categories[keyword].add(skill_id)

 return skill

def find_skills_by_keyword(self, keyword: str) -> List[Skill]:
 """Find skills by keyword"""
 skill_ids = self.skill_categories.get(keyword, set())
 return [self.skills[skill_id] for skill_id in skill_ids if skill_id in self.skills]

def get_skill(self, skill_id: str) -> Optional[Skill]:
 """Get skill by ID"""
 return self.skills.get(skill_id)

Global skill registry
skill_registry = SkillRegistry()
'''

```

## 1.2 Task Management System

```

'''python
a2a/task_management.py
import uuid
from datetime import datetime
from typing import Dict, List, Optional, Any
from dataclasses import dataclass, field
from enum import Enum

@dataclass
class Task:
 """Complete task definition with tracking"""
 task_id: str # Format: TSK-{flow}-{uuid}
 task_name: str
 date_created: datetime
 time_created: str
 description: str
 sender_agent_id: str
 destination_agent_id: str
 flow_tag: FlowDirection
 status: TaskStatus
 priority: int = 1 # 1-10, 10 being highest

```

```

dependencies: List[str] = field(default_factory=list) # task_ids this task depends on
required_skills: List[str] = field(default_factory=list) # skill_ids required
input_data: Dict[str, Any] = field(default_factory=dict)
output_data: Dict[str, Any] = field(default_factory=dict)
error_details: Optional[str] = None
retry_count: int = 0
max_retries: int = 3
timeout_seconds: int = 300
metadata: Dict[str, Any] = field(default_factory=dict)

Timestamps
started_at: Optional[datetime] = None
completed_at: Optional[datetime] = None
last_updated: datetime = field(default_factory=datetime.utcnow)

```

```
class TaskManager:
```

```
 """Manages task lifecycle and tracking"""
```

```
 def __init__(self):
```

```
 self.tasks: Dict[str, Task] = {}
```

```
 self.agent_tasks: Dict[str, List[str]] = {} # agent_id -> list of task_ids
```

```
 self.task_dependencies: Dict[str, List[str]] = {} # task_id -> list of blocking task_ids
```

```
 def generate_task_id(self, flow_direction: FlowDirection) -> str:
```

```
 """Generate unique task ID with flow direction"""
```

```
 unique_id = str(uuid.uuid4())[:8]
```

```
 flow_code = flow_direction.value[:3].upper()
```

```
 return f"TSK-{flow_code}-{unique_id}"
```

```
 def create_task(self, task_name: str, description: str, sender_agent_id: str,
```

```
 destination_agent_id: str, flow_direction: FlowDirection,
```

```
 priority: int = 1, dependencies: List[str] = None,
```

```
 required_skills: List[str] = None, input_data: Dict = None,
```

```
 timeout_seconds: int = 300) -> Task:
```

```
 """Create a new task"""
```

```
 task_id = self.generate_task_id(flow_direction)
```

```
 current_time = datetime.utcnow()
```

```
 task = Task(
```

```
 task_id=task_id,
```

```
 task_name=task_name,
```

```
 date_created=current_time.date(),
```

```
 time_created=current_time.strftime("%H:%M:%S"),
```

```

 description=description,
 sender_agent_id=sender_agent_id,
 destination_agent_id=destination_agent_id,
 flow_tag=flow_direction,
 status=TaskStatus.CREATED,
 priority=priority,
 dependencies=dependencies or [],
 required_skills=required_skills or [],
 input_data=input_data or {},
 timeout_seconds=timeout_seconds,
 metadata={
 "created_timestamp": current_time.isoformat(),
 "version": "1.0"
 }
)

```

```

Store task
self.tasks[task_id] = task

```

```

Track agent tasks
if sender_agent_id not in self.agent_tasks:
 self.agent_tasks[sender_agent_id] = []
self.agent_tasks[sender_agent_id].append(task_id)

```

```

if destination_agent_id not in self.agent_tasks:
 self.agent_tasks[destination_agent_id] = []
self.agent_tasks[destination_agent_id].append(task_id)

```

```

Handle dependencies
if dependencies:
 self.task_dependencies[task_id] = dependencies

```

```

return task

```

```

def update_task_status(self, task_id: str, status: TaskStatus,
 output_data: Dict = None, error_details: str = None) -> bool:
 """Update task status and progress"""

```

```

 if task_id not in self.tasks:
 return False

```

```

 task = self.tasks[task_id]
 task.status = status
 task.last_updated = datetime.utcnow()

```

```

if status == TaskStatus.IN_PROGRESS and not task.started_at:
 task.started_at = datetime.utcnow()

elif status in [TaskStatus.SUCCESS, TaskStatus.FAILURE, TaskStatus.COMPLETE]:
 task.completed_at = datetime.utcnow()

if output_data:
 task.output_data = output_data

if error_details:
 task.error_details = error_details
 if status != TaskStatus.FAILURE:
 task.retry_count += 1

return True

def get_task(self, task_id: str) -> Optional[Task]:
 """Get task by ID"""
 return self.tasks.get(task_id)

def get_agent_tasks(self, agent_id: str, status: TaskStatus = None) -> List[Task]:
 """Get tasks for a specific agent"""
 task_ids = self.agent_tasks.get(agent_id, [])
 tasks = [self.tasks[task_id] for task_id in task_ids if task_id in self.tasks]

 if status:
 tasks = [task for task in tasks if task.status == status]

 return tasks

def can_task_proceed(self, task_id: str) -> bool:
 """Check if task dependencies are satisfied"""
 if task_id not in self.task_dependencies:
 return True

 dependencies = self.task_dependencies[task_id]
 for dep_task_id in dependencies:
 dep_task = self.tasks.get(dep_task_id)
 if not dep_task or dep_task.status not in [TaskStatus.SUCCESS,
TaskStatus.COMPLETE]:
 return False

 return True

```

```

def get_task_chain(self, task_id: str) -> List[Task]:
 """Get the complete chain of tasks leading to this one"""
 chain = []
 current_task_id = task_id

 while current_task_id in self.tasks:
 task = self.tasks[current_task_id]
 chain.append(task)
 # Follow sender chain (simplified - in reality would use proper tracing)
 if task.sender_agent_id and task.sender_agent_id in self.agent_tasks:
 # Find previous task from same sender
 sender_tasks = self.get_agent_tasks(task.sender_agent_id)
 previous_tasks = [t for t in sender_tasks if t.completed_at and t.completed_at <
task.started_at]
 if previous_tasks:
 # Get most recent previous task
 previous_tasks.sort(key=lambda x: x.completed_at, reverse=True)
 current_task_id = previous_tasks[0].task_id
 else:
 break
 else:
 break

 return list(reversed(chain)) # Return in chronological order

Global task manager
task_manager = TaskManager()
'''

```

### 1.3 Hierarchical Agent ID System

```

```python
# a2a/agent_hierarchy.py
import uuid
from typing import Dict, List, Optional
from dataclasses import dataclass, field

@dataclass
class AgentIdentity:
    """Hierarchical agent identity system"""
    agent_id: str # Full hierarchical ID
    base_agent_id: str # Root agent ID without sub-agent tags
    parent_agent_id: Optional[str]

```

```
agent_type: str # primary, sba (sub-agent), supervisor
generation: int # 0 for root, 1 for first-level sub-agents, etc.
created: str
capabilities: List[str]
```

```
def is_sub_agent(self) -> bool:
    return self.agent_type == "sba"
```

```
def get_root_agent_id(self) -> str:
    """Get the root agent ID (remove all .sba. segments)"""
    parts = self.agent_id.split('.')
    root_parts = [part for part in parts if part != 'sba']
    return '.'.join(root_parts[:2]) # Return format: type_id
```

```
class AgentHierarchy:
    """Manages hierarchical agent relationships"""
```

```
def __init__(self):
    self.agents: Dict[str, AgentIdentity] = {}
    self.children: Dict[str, List[str]] = {} # parent_agent_id -> list of child_agent_ids
```

```
def generate_agent_id(self, base_name: str, agent_type: str,
    parent_agent_id: Optional[str] = None) -> str:
    """Generate hierarchical agent ID"""
```

```
    if parent_agent_id:
        # Sub-agent generation
        if not parent_agent_id.endswith('.sba'):
            # First-level sub-agent
            return f"{parent_agent_id}.sba.{base_name}"
        else:
            # Nested sub-agent
            return f"{parent_agent_id}.{base_name}"
    else:
        # Primary agent
        unique_id = str(uuid.uuid4())[:8]
        return f"{base_name}_{unique_id}"
```

```
def register_agent(self, base_name: str, agent_type: str,
    parent_agent_id: Optional[str] = None,
    capabilities: List[str] = None) -> AgentIdentity:
    """Register a new agent in the hierarchy"""
```

```
    agent_id = self.generate_agent_id(base_name, agent_type, parent_agent_id)
```

```

# Calculate generation
if parent_agent_id:
    parent_agent = self.agents.get(parent_agent_id)
    generation = parent_agent.generation + 1 if parent_agent else 1
else:
    generation = 0

agent_identity = AgentIdentity(
    agent_id=agent_id,
    base_agent_id=base_name,
    parent_agent_id=parent_agent_id,
    agent_type=agent_type,
    generation=generation,
    created=datetime.utcnow().isoformat(),
    capabilities=capabilities or []
)

self.agents[agent_id] = agent_identity

# Update parent-child relationships
if parent_agent_id:
    if parent_agent_id not in self.children:
        self.children[parent_agent_id] = []
    self.children[parent_agent_id].append(agent_id)

return agent_identity

def get_agent(self, agent_id: str) -> Optional[AgentIdentity]:
    """Get agent identity by ID"""
    return self.agents.get(agent_id)

def get_sub_agents(self, parent_agent_id: str) -> List[AgentIdentity]:
    """Get all sub-agents of a parent agent"""
    child_ids = self.children.get(parent_agent_id, [])
    return [self.agents[child_id] for child_id in child_ids if child_id in self.agents]

def get_agent_lineage(self, agent_id: str) -> List[AgentIdentity]:
    """Get the complete lineage from root to this agent"""
    lineage = []
    current_agent_id = agent_id

    while current_agent_id in self.agents:
        agent = self.agents[current_agent_id]

```

```

        lineage.append(agent)
        if agent.parent_agent_id:
            current_agent_id = agent.parent_agent_id
        else:
            break

    return list(reversed(lineage)) # Root first

def is_descendant_of(self, agent_id: str, potential_ancestor_id: str) -> bool:
    """Check if an agent is a descendant of another"""
    lineage = self.get_agent_lineage(agent_id)
    ancestor_ids = [agent.agent_id for agent in lineage]
    return potential_ancestor_id in ancestor_ids

def get_capability_inheritance(self, agent_id: str) -> List[str]:
    """Get all capabilities inherited from parent agents"""
    lineage = self.get_agent_lineage(agent_id)
    all_capabilities = []

    for agent in lineage:
        all_capabilities.extend(agent.capabilities)

    return list(set(all_capabilities)) # Remove duplicates

# Global agent hierarchy
agent_hierarchy = AgentHierarchy()
...

```

1.4 A2A Message Protocol with Agent Cards

```

```python
a2a/message_protocol.py
from datetime import datetime
from typing import Dict, Any, Optional
from dataclasses import dataclass, field
import json

@dataclass
class A2AMessage:
 """Enhanced A2A message with agent card support"""
 message_id: str
 sender_agent_id: str
 recipient_agent_id: str
 message_type: str # TASK_REQUEST, TASK_RESPONSE, AGENT_CARD, HEARTBEAT

```

```

task_id: Optional[str] = None
payload: Dict[str, Any] = field(default_factory=dict)
sender_agent_card: Optional[Dict] = None # Sender's agent card
required_skills: List[str] = field(default_factory=list)
timestamp: datetime = field(default_factory=datetime.utcnow)
correlation_id: Optional[str] = None # For request-response matching
priority: int = 1
expiration: Optional[datetime] = None

```

```

def to_dict(self) -> Dict[str, Any]:
 """Convert message to dictionary for transmission"""
 return {
 "message_id": self.message_id,
 "sender_agent_id": self.sender_agent_id,
 "recipient_agent_id": self.recipient_agent_id,
 "message_type": self.message_type,
 "task_id": self.task_id,
 "payload": self.payload,
 "sender_agent_card": self.sender_agent_card,
 "required_skills": self.required_skills,
 "timestamp": self.timestamp.isoformat(),
 "correlation_id": self.correlation_id,
 "priority": self.priority,
 "expiration": self.expiration.isoformat() if self.expiration else None
 }

```

@classmethod

```

def from_dict(cls, data: Dict[str, Any]) -> 'A2AMessage':
 """Create message from dictionary"""
 return cls(
 message_id=data["message_id"],
 sender_agent_id=data["sender_agent_id"],
 recipient_agent_id=data["recipient_agent_id"],
 message_type=data["message_type"],
 task_id=data.get("task_id"),
 payload=data.get("payload", {}),
 sender_agent_card=data.get("sender_agent_card"),
 required_skills=data.get("required_skills", []),
 timestamp=datetime.fromisoformat(data["timestamp"].replace('Z', '+00:00')),
 correlation_id=data.get("correlation_id"),
 priority=data.get("priority", 1),
 expiration=datetime.fromisoformat(data["expiration"].replace('Z', '+00:00')) if
data.get("expiration") else None
)

```

```
class A2AProtocol:
```